GUÍA TÉCNICA · PUESTA EN PRODUCCIÓN

Poner VIRF en Databricks

Runbook para tu primera vez en producción · conexión a tablas SQL de la nube (Unity Catalog)
Generado el 24/09/2026 · basado en el código real del repositorio

Poner VIRF en Databricks — guía para tu primera vez en producción

> **Para quién es esta guía:** alguien que sabe programar pero **nunca ha llevado un proyecto a producción** y no conoce Databricks a fondo. Vamos despacio: primero se entiende el terreno, luego se evalúa si es factible, y por último se hace paso a paso.
>
> **Meta:** que VIRF corra en Databricks conectado a las **tablas SQL de la nube** (Unity Catalog) para (A) ejecutar los **controles de datos** y (B) **cuestionar** y emitir el informe.
>
> **Fecha:** 2026-09-24 · Basado en el código real del repositorio.

PARTE I — Entender el terreno antes de tocar nada

1. ¿Qué significa "poner en producción"? (en cristiano)

Hoy VIRF corre **en tu computadora** ("local"): tú lo arrancas, lee archivos de tu disco, usa tu `.env` con las claves, y lo ves en `localhost`. Eso es un **prototipo**: funciona para ti, cuando tú estás delante.

Producción es lo contrario: el sistema corre **en un servidor de la empresa**, sin ti delante, de forma **repetible, segura y gobernada**. Cambian cinco cosas:

	Local (hoy)	Producción (meta)
Dónde corre	Tu PC	Un clúster de Databricks
De dónde saca los datos	CSV en tu disco	Tablas SQL de la nube (Unity Catalog)
Las claves/secretos	Archivo `.env`	Una bóveda cifrada (secret scope)
Quién lo arranca	Tú, a mano	Un "Job" programado (o un botón)
Quién puede verlo/usarlo	Solo tú	Con permisos y registro de quién hizo qué

Poner en producción **no es "subir el código y ya"**. Es adaptar esas 5 cosas. La buena noticia: VIRF ya está diseñado para que ese salto sea corto (lo verás en la §4).

2. ¿Qué vamos a lograr exactamente? Los dos flujos de VIRF

VIRF hace dos cosas distintas. **Entiéndelas por separado**, porque solo una se conecta a las tablas:

- **Flujo A — Controles de datos (lo tabular).** Revisa datos: "¿el LCR nunca baja de 100%?", "¿el backtesting de Kupiec pasa?", "¿hay duplicados?". Cada control es una funcioncita Python que recibe una tabla (un `DataFrame`) y devuelve *Cumple / No cumple / …*. **Esto es lo que se conecta a las tablas SQL.** Hoy lee un CSV; mañana leerá una tabla de la nube. Los controles **no cambian**: solo cambia de dónde llega la tabla.
- **Flujo B — Validación con cuestionamiento (lo documental).** Lee la **documentación** del modelo (manuales PDF, código, actas), la contrasta con literatura regulatoria (el "RAG") y **le hace preguntas críticas a una IA** para producir el informe. Aquí la entrada son **documentos**, no tablas.

> **Regla de oro:** *tablas SQL → Flujo A. Documentos → Flujo B.* Puedes montar solo A, solo B, o los dos. **Para tu primera vez, monta solo el Flujo A** (es más simple y no depende de que aprueben sacar datos a la IA — ver §5).

3. Conceptos de Databricks, explicados con analogías

No necesitas ser experto; con estos ocho conceptos sigues toda la guía.

- **Workspace** = el "Databricks" de tu empresa: la página web donde entras a todo.
- **Cluster** = la computadora en la nube que ejecuta tu código Python. La enciendes, corre, la apagas. (Un **SQL Warehouse** es lo mismo pero solo para consultas SQL.)
- **Notebook** = como un cuaderno de Jupyter dentro de Databricks: celdas de código que corres una a una. Es donde probarás todo al principio.

- **Unity Catalog (UC)** = el "catálogo de tablas" gobernado de la empresa. Las tablas se nombran con **tres niveles**: `catalogo.esquema.tabla` (p. ej. `riesgo.var.posiciones_2027`). Piensa: *archivador.cajón.carpeta*.
- **Volume** = una carpeta de archivos gobernada por UC (`/Volumes/riesgo/virf/…`). Ahí guardaremos el corpus del RAG, la documentación y las salidas. (**DBFS** es una versión vieja del mismo concepto, `/dbfs/…`; el RAG ya la usa por dentro.)
- **Secret scope** = una **caja fuerte** para claves. En vez de escribir la clave del LLM en el código (peligroso), la guardas ahí y el código la pide con `dbutils.secrets.get`.
- **Job** = una tarea que Databricks corre **sola** (a una hora, o cuando la disparas), con parámetros. Es lo que convierte "yo corro el notebook" en "esto corre solo cada mes".
- **`spark` y `dbutils`** = dos "ayudantes" que ya existen dentro de cualquier notebook Databricks (no los instalas). `spark` lee tablas; `dbutils` maneja secretos, archivos y parámetros.

PARTE II — ¿Es factible? (evaluación honesta)

4. Evaluación de factibilidad, dimensión por dimensión

> Escala: 🟢 alta / 🟡 media / 🔴 baja. "Por qué" = la razón real, del código.

Dimensión	Nivel	Por qué
Técnica	🟢 alta	El RAG es **léxico y local** (BM25+TF-IDF): sin base vectorial, sin HuggingFace, nada que montar ni sacar a la nube. Los controles ya son funciones sobre `pandas.DataFrame`. Y ya hay soporte Databricks a medias (`scripts/rebuild_rag_databricks.py`; el índice se guarda en `/dbfs` solo). El único trabajo nuevo real es **un adaptador de datos** (§Fase 4).
De datos	🟡 media-alta	Depende de que las tablas UC existan y de que sus **columnas** coincidan con lo que cada control espera (o de ajustar los `params`, como se hace hoy para un CSV). Riesgo real: **tablas enormes** — traer todo a memoria (`toPandas()`) puede reventar; se resuelve filtrando/agregando (§14).
Seguridad / cumplimiento	🟡 media	**El punto crítico.** El Flujo A (controles) corre **dentro** del clúster: cero datos salen → viable sin permisos especiales. El Flujo B (cuestionamiento) **envía evidencia a la IA** (Azure OpenAI/Anthropic) → eso es *egress* y necesita aprobación. El enmascarado de PII y la anti-inyección ya reducen el riesgo, pero la decisión es de tu área de seguridad.
Coste	🟡 media	Dos costes: **cómputo** Databricks (controlable: clúster pequeño, apagado automático) y **tokens de IA** (solo Flujo B; controlable con lotes y con el entailment apagado por defecto).
Esfuerzo / habilidades	🟡 media	No necesitas ser experto, pero sí **alguien con acceso admin a Databricks** para provisionar (o pedírselo a plataforma). La curva para un primerizo es real pero corta con esta guía.
Mantenimiento	🟢 alta	Reproducible (versiones **fijadas** en `requirements.txt`), el RAG se reconstruye con un Job mensual, y los controles viven versionados en Git.

Veredicto

🟢 **FACTIBLE.** Y hay una jugada clara para minimizar riesgo:

- **El Flujo A (controles sobre tablas) es de baja fricción, alto valor y NO requiere sacar datos a ninguna IA.** Empieza por ahí. En ~1–2 semanas tienes controles corriendo sobre tablas reales.
- **El Flujo B (cuestionamiento) es igual de viable técnicamente**, pero **depende de una decisión de seguridad** (permitir egress al proveedor de IA). Arránquelo *después* de esa aprobación.

5. Qué podría bloquearlo (y qué hacer)

Posible bloqueo	Señal	Qué hacer
Unity Catalog no habilitado	No puedes hacer `catalogo.esquema.tabla`	Pedir a plataforma que habilite UC (o usar el esquema legacy `hive_metastore`).
No aprueban el egress al LLM	Seguridad dice "no sale data a un tercero"	Desplegar **solo el Flujo A**; el B queda en pausa o se evalúa un LLM interno.

Posible bloqueo	Señal	Qué hacer
Tablas gigantes	`toPandas()` se cuelga / OOM	Filtrar por ventana y **agregar en Spark** antes de pasar a pandas (§14).
Nadie con permisos admin	No puedes crear cluster/secret scope/Volume	Conseguir un "sponsor" de plataforma; casi todo lo hace él una vez.
No hay remoto Git	No puedes usar "Repos"	Subir el código al Volume (opción 2 de la Fase 1).

6. Ruta recomendada para tu PRIMERA vez (no te compliques)

No intentes todo de golpe. Orden sugerido para un primerizo:

1. **Prueba de vida** (Fase 0–2): entra a Databricks, crea un notebook, lee 5 filas de una tabla con SQL. *Meta: "sé leer una tabla".*
2. **Un control, una tabla, a mano** (Fase 4–6, versión mínima): en un notebook, trae una tabla a pandas y corre **un** control. *Meta: "vi un Cumple/No cumple real".*
3. **Todos los controles de un dominio** (Fase 6 completa) → guarda el cuadro en una tabla.
4. **Automatiza con un Job** (Fase 8).
5. Solo entonces, si seguridad aprueba, monta el **Flujo B** (Fases 5 y 7).

Cada nivel funciona por sí solo. Si te quedas en el 3, ya tienes valor real en producción.

PARTE III — Los pasos, en detalle

> Cada fase trae: **Por qué existe · Qué haces · Qué vas a ver · Cómo saber que salió bien · Error típico de primerizo · A quién pedir ayuda.**

FASE 0 — Prerrequisitos y la decisión de seguridad

Por qué existe: en producción no "instalas y ya"; necesitas accesos y una decisión de seguridad *antes* de escribir código, o te bloquearás a mitad.

Qué haces: - [ ] Confirma que hay **Unity Catalog** y un esquema con las tablas de riesgo. - [ ] Consigue permiso de **lectura** (`SELECT`) sobre esas tablas. - [ ] Consigue un **cluster** (DBR 14.3 LTS o superior; "single node" sobra para empezar). - [ ] Crea un **Volume** para archivos: `sql CREATE VOLUME IF NOT EXISTS riesgo.virf.datos; -- queda en /Volumes/riesgo/virf/datos/` - [ ] Instala y conecta el **Databricks CLI** en tu PC (para subir cosas): `bash pip install databricks-cli databricks configure --token # te pide la URL del workspace y un token (lo generas en la UI)` - [ ] **Decisión de seguridad:** ¿el clúster puede llamar a Azure OpenAI/Anthropic? Escríbelo y consíguelo por correo/ticket. (Si la respuesta es "no", sigues con el Flujo A.)

Qué vas a ver: acceso al workspace; un `SELECT ... LIMIT 5` que devuelve filas.

Cómo saber que salió bien: `databricks fs ls dbfs:/` responde sin error.

Error típico de primerizo: pedir permisos "cuando haga falta". Pídelos **el día 1**: son lo que más tarda.

A quién pedir ayuda: equipo de **plataforma/datos** (accesos, cluster, UC) y **seguridad** (egress).

Tiempo: 1–3 días (casi todo es esperar accesos).

FASE 1 — Subir el código a Databricks

Por qué existe: el código está en tu PC; el clúster no lo ve. Hay que llevarlo allá.

Qué haces — elige un camino:

- **Opción 1 (recomendada): "Repos" (carpeta Git).** Databricks se conecta a un repo Git y lo trae. *Requiere un remoto Git* (hoy tu repo es local). Pasos: `bash git remote add origin <URL-de-tu-repo> # GitHub/Azure DevOps/GitLab git push -u origin main` Luego en la web: **Workspace → Repos → Add Repo → pega la URL**. Para traer cambios futuros, botón **"Pull"**.
- **Opción 2 (sin Git): subir los archivos al Volume.** `bash databricks fs cp -r . dbfs:/Volumes/riesgo/virf/codigo/` Y en el notebook, decirle a Python dónde está el código: `python import sys; sys.path.insert(0, "/Volumes/riesgo/virf/codigo")`

Qué vas a ver: la carpeta del proyecto dentro del workspace o del Volume.

Cómo saber que salió bien: en un notebook, `import modulos.repositorio_controles` no falla.

Error típico de primerizo: olvidar el `sys.path.insert(...)` → "ModuleNotFoundError". El proyecto asume que se ejecuta **desde su raíz**.

Tiempo: 0.5–1 día.

FASE 2 — Instalar las librerías (con las versiones exactas)

Por qué existe: el clúster viene "pelado". Y **la versión importa**: el índice del RAG depende de `rank_bm25` y `scikit-learn`; si cambian de versión, cambia el resultado. Por eso `requirements.txt` las fija con `==`.

Qué haces: en la primera celda del notebook:

```
%pip install -r /Workspace/Repos/<tu-usuario>/virf/requirements.txt
dbutils.library.restartPython()   # OBLIGATORIO: reinicia Python para tomar lo instalado
```

Qué vas a ver: pip instalando; luego el kernel se reinicia.

Cómo saber que salió bien: `import rank_bm25, sklearn` sin error.

Error típico de primerizo: olvidar `restartPython()` → "ya instalé pero sigue fallando el import". Es porque Python seguía con el entorno viejo.

Tiempo: 0.5–1 día.

FASE 3 — Guardar las claves de forma segura (secret scope)

Por qué existe: en producción **nunca** se ponen claves en el código ni en un `.env` subido. Se guardan cifradas y el código las pide en runtime.

Qué haces (una vez, desde tu PC):

```
databricks secrets create-scope virf
databricks secrets put --scope virf --key azure_openai_key
databricks secrets put --scope virf --key azure_openai_endpoint
databricks secrets put --scope virf --key azure_openai_deployment
```

Y en el notebook, **antes** de importar el motor:

```
import os
os.environ["IA_PROVEEDOR"]            = "azure_openai"
os.environ["AZURE_OPENAI_API_KEY"]    = dbutils.secrets.get("virf", "azure_openai_key")
os.environ["AZURE_OPENAI_ENDPOINT"]   = dbutils.secrets.get("virf", "azure_openai_endpoint")
os.environ["AZURE_OPENAI_DEPLOYMENT"] = dbutils.secrets.get("virf", "azure_openai_deployment")
```

> Esto **solo hace falta para el Flujo B** (el que usa la IA). Si empiezas por el Flujo A, puedes saltarte esta fase.

Qué vas a ver: nada visible (los secretos no se imprimen).

Cómo saber que salió bien (prueba real de la IA):

```
from config.ia import prueba_de_humo
print(prueba_de_humo())   # -> {'ok': True, 'modelo': '...', 'json_ok': True, ...}
```

Error típico de primerizo: intentar imprimir el secreto para "verificar" → Databricks lo tapa con `[REDACTED]` (es a propósito). Un `401` = clave/endpoint mal; un *timeout* = egress bloqueado (vuelve a §5).

Tiempo: 0.5–1 día.

FASE 4 — El adaptador a las tablas (el corazón del trabajo)

Por qué existe: hoy VIRF lee datasets de una **carpeta de CSV** (`cargar_datasets(carpeta)`). En Databricks los datos están en **tablas**. Hace falta una función que lea de tablas **pero devuelva lo mismo** (`{nombre: DataFrame}`), para que los controles no se enteren del cambio.

Cómo funciona por dentro: `spark.read.table("cat.esq.tabla")` te da una tabla "distribuida" (grande, perezosa). `.toPandas()` la trae a memoria como el `DataFrame` de siempre. Un **mapa** `{nombre_lógico: tabla_física}` separa el nombre que usan los controles del nombre real de la tabla.

Qué haces: añadir a `modulos/motores_datos.py`:

```
def cargar_datasets_uc(mapa: dict, *, spark=None,
                       limite_filas: int | None = None,
                       where: dict | None = None) -> dict:
    """Lee datasets desde tablas Unity Catalog.
       mapa  = {"principal": "riesgo.var.posiciones_2027", ...}
       where = {"pnl": "fecha >= '2027-01-01'"}   # filtro que se empuja a la tabla
       Devuelve {nombre: pandas.DataFrame} — igual que cargar_datasets()."""
    if spark is None:
        from pyspark.sql import SparkSession
        spark = SparkSession.builder.getOrCreate()
    out = {}
    for nombre, tabla in mapa.items():
        sdf = spark.read.table(tabla)
        if where and nombre in where:
            sdf = sdf.where(where[nombre])      # filtra en la tabla, no en memoria
        if limite_filas:
            sdf = sdf.limit(limite_filas)
        out[nombre] = sdf.toPandas()
    return out
```

Qué vas a ver: un dict con DataFrames que puedes `display(...)`.

Cómo saber que salió bien:

```
d = cargar_datasets_uc({"principal": "riesgo.var.posiciones_2027"}, limite_filas=100)
display(d["principal"])   # 100 filas, las columnas de tu tabla
```

Error típico de primerizo: poner solo `tabla` en vez de `catalogo.esquema.tabla` → "Table or view not found". Siempre los **tres niveles**.

Tiempo: 2–4 días (código + decidir el mapa por dominio + pruebas). *Es el grueso.*

FASE 5 — Preparar el RAG (solo si haces el Flujo B)

Por qué existe: el cuestionamiento se apoya en literatura regulatoria indexada (el RAG). Hay que subir el corpus y construir el índice **en el clúster** (es local, no sale nada a la nube).

Qué haces:

```
databricks fs cp -r inputs/rag dbfs:/Volumes/riesgo/virf/datos/inputs/rag
```

```
%sh cd /Workspace/Repos/<tu>/virf && python scripts/rebuild_rag_databricks.py --dominio all
```

Cómo saber que salió bien:

```
%sh cd /Workspace/Repos/<tu>/virf && python scripts/rebuild_rag_databricks.py --status
# cada dominio (LaR, VaR, ALM, shared) con chunks > 0
```

Detalle útil: prográmalo como **Job mensual** (el script ya sugiere cron `0 2 1 * *`).

Tiempo: 0.5–1 día.

FASE 6 — Correr los controles sobre las tablas (Flujo A, ¡el objetivo!)

Por qué existe: es *el* resultado que buscas — controles de calidad/negocio corriendo sobre datos reales de la nube.

Cómo funciona: `cargar_controles(dominio)` trae los controles del dominio + los `shared`. Cada uno se ejecuta con la tabla y devuelve `Cumple/No cumple/…`. Se junta todo en un cuadro y (opcional) se guarda como tabla para consumir desde SQL/BI.

Qué haces — un notebook completo:

```
import sys; sys.path.insert(0, "/Workspace/Repos/<tu>/virf")
from modulos.motores_datos import cargar_datasets_uc
from modulos.repositorio_controles import cargar_controles
import pandas as pd

# 1) traer las tablas
datasets = cargar_datasets_uc({
    "principal": "riesgo.var.posiciones_2027",
    "pnl":       "riesgo.var.pnl_diario",
}, where={"pnl": "fecha >= '2027-01-01'"})

# 2) ejecutar los controles del dominio (VaR / ALM / LaR)
filas = []
for c in cargar_controles("VaR")[0]:
    ns = {}
    exec(compile(c["fuente"], c["archivo"], "exec"), ns)   # el control es código propio
    params = dict(c["meta"].get("params", {}))
    df = datasets.get(params.get("dataset") or "principal", datasets["principal"])
    try:
        res = ns["control"](df, datasets, params)
    except Exception as e:
        res = {"pct": None, "decision": "No evaluable", "diagnostico": f"error: {e}"}
    filas.append({"control": c["nombre"], "dimension": c["meta"].get("dimension"), **res})

cuadro = pd.DataFrame(filas)
display(cuadro)

# 3) (opcional) guardar el resultado como tabla, para consumir desde SQL
spark.createDataFrame(cuadro).write.mode("overwrite").saveAsTable("riesgo.virf.resultado_controles_var")
```

Qué vas a ver: una tabla con una fila por control: `Cumple/Observado/No cumple/No evaluable`, su `%` y un diagnóstico.

Cómo saber que salió bien: las decisiones tienen sentido y la tabla `riesgo.virf.resultado_controles_var` se puede consultar por SQL.

Error típico de primerizo: que **casi todo** salga "No evaluable". Significa que los nombres de columna que espera el control (`params`) no son los de tu tabla. Ábrelos y ajústalos a tu esquema (exactamente igual que se haría con un CSV).

Tiempo: 1–2 días.

FASE 7 — Cuestionar y generar el informe (Flujo B; requiere el OK de seguridad)

Por qué existe: es la parte que "piensa": contrasta la documentación con la literatura y **hace preguntas críticas** con la IA para emitir el informe.

Cómo funciona: `agente.py` orquesta; `motor_ia.evaluar_actividad` arma el prompt (documentación + RAG + calibración) y consulta la IA. Antes de salir, la evidencia pasa por **enmascarado de PII** y **anti-inyección** (ya activos). El RAG se consulta **local**.

Qué haces — la forma simple (como script):

```
%sh cd /Workspace/Repos/<tu>/virf && \
    python agente.py --raiz-evidencia /Volumes/riesgo/virf/datos/evidencia/<modelo> \
                     --dominio-rag ALM --titulo "Modelo ALM Prepago" --tier II
# revisa `python agente.py --help` para todos los flags
```

La documentación del modelo (PDFs, código) va en el Volume; el informe `.docx` sale a `outputs/` (apúntalo a un Volume para descargarlo).

Cómo saber que salió bien: aparece el `.docx`; en el JSON de resultados ves los campos `pii_enmascarada`, `inyeccion_detectada` y (si lo activaste) `entailment`.

Error típico de primerizo: el filtro de contenido de Azure bloquea vocabulario de riesgo ("pérdida", "default"…). El propio cliente te dice qué categoría fue y las 3 salidas (ajustar el filtro del deployment, trocear, o `VIRF_MAX_CTX`).

Tiempo: 0.5–1 día.

FASE 8 — Automatizar con un Job

Por qué existe: "yo corro el notebook" no es producción. Un **Job** lo corre solo, con parámetros, a una hora, y deja registro.

Cómo funciona: el Job ejecuta tu notebook con **parámetros** que el notebook lee con `dbutils.widgets`.

```
# arriba del notebook
dbutils.widgets.text("dominio", "ALM")
dominio = dbutils.widgets.get("dominio")
```

```
# crear el Job (desde tu PC)
databricks jobs create --json '{
  "name": "VIRF · controles ALM",
  "tasks": [{
    "task_key": "controles",
    "notebook_task": {"notebook_path": "/Repos/<tu>/virf/notebooks/controles",
                      "base_parameters": {"dominio": "ALM"}},
    "new_cluster": {"spark_version": "14.3.x-scala2.12", "num_workers": 0}
  }],
  "schedule": {"quartz_cron_expression": "0 0 6 * * ?", "timezone_id": "America/Lima"}
}'
```

Cómo saber que salió bien: el Job aparece en **Workflows**, lo corres a mano una vez y deja el cuadro/informe en el Volume o la tabla.

Tiempo: 2–3 días (con salidas a Volume y permisos).

FASE 9 — La interfaz web (opcional)

- **A) Sin web (recomendado al inicio):** el notebook *es* el entregable. **0 días.**
- **B) Databricks App:** hospedar `web/app.py` (FastAPI) dentro de Databricks. **3–5 días.**
- **C) Web fuera de Databricks:** dejar la SPA en un servidor propio que consulte el SQL Warehouse con `databricks-sql-connector`. **2–4 días.**

PARTE IV — Cierre

13. Cómo fluye una corrida, de principio a fin

1. Un **Job** arranca (a una hora o a mano) con el dominio/modelo como parámetro.
2. **Flujo A:** `cargar_datasets_uc(...)` lee las tablas → corre los controles → guarda el cuadro en una tabla de resultados.
3. **Flujo B (si aplica):** lee la documentación del Volume, consulta el RAG local, enmascara PII, y **cuestiona** con la IA → genera informe `.docx` + acta `.xlsx` + JSON.
4. Las **salidas** quedan en `/Volumes/riesgo/virf/salidas/` y, si quieres, como tabla.
5. **Unity Catalog** guarda el linaje: qué se leyó, quién corrió, qué se produjo.

Lo único que sale del clúster son las llamadas a la IA del paso 3 (con la evidencia ya enmascarada). Leer tablas y usar el RAG son operaciones internas.

14. Riesgos y cómo mitigarlos

Riesgo	Por qué pasa	Mitigación
Sacar datos a la IA	El cuestionamiento envía evidencia al proveedor	Azure OpenAI por red privada; PII masking (ON); aprobar con seguridad; o solo Flujo A
Tabla gigante → memoria	`toPandas()` trae todo al driver	Filtrar por ventana (`where`) y **agregar en Spark** antes de pasar a pandas
Claves expuestas	Ponerlas en el código	Secret scopes + `dbutils.secrets.get`
Proxy corporativo (TLS)	El SDK no confía en la CA propia	`VIRF_CA_BUNDLE` apuntando al certificado
Resultados cambian entre corridas	Versiones de librería distintas	`requirements.txt` con versiones fijas
Coste de IA	Muchas llamadas	Entailment apagado por defecto; correr por lotes

15. Cuánto tarda (y por qué)

> Días-persona, 1 ingeniero con accesos listos. El rango alto refleja fricción corporativa (permisos, red, seguridad), que para un primerizo suele ser **la mayor parte del tiempo real**.

Fase	Días
0 · Prerrequisitos + decisión de seguridad	1 – 3
1 · Subir código	0.5 – 1
2 · Dependencias	0.5 – 1
3 · Secretos + IA (solo Flujo B)	0.5 – 1
4 · **Adaptador a tablas**	2 – 4
5 · RAG (solo Flujo B)	0.5 – 1

Fase	Días
6 · Controles sobre tablas (Flujo A)	1 – 2
7 · Cuestionamiento (Flujo B)	0.5 – 1
8 · Job + gobierno	2 – 3
9 · Web (opcional)	0 / 3 – 5
— · Pruebas E2E + estabilizar	2 – 3

- **Solo Flujo A (controles sobre tablas), tu primer objetivo realista: ≈ 1–2 semanas.**
- **Producción completa (A + B, con Jobs, sin web): ≈ 3–4 semanas.**
- **Con web: ≈ 4–6 semanas.**

> Nota para primerizos: si es tu primera producción, **suma un colchón** de tiempo por la curva de aprendizaje y las idas y vueltas con plataforma/seguridad. Es normal.

16. Checklist (en orden)

1. [ ] Accesos, cluster, Volume, CLI, y **decisión de egress** (Fase 0).
2. [ ] Código en Databricks (Repo o Volume) e `import` OK (Fase 1).
3. [ ] `requirements.txt` instalado + `restartPython()` (Fase 2).
4. [ ] `cargar_datasets_uc()` leyendo **una** tabla real (Fase 4).
5. [ ] **Un** dominio de controles corriendo sobre esa tabla (Fase 6).
6. [ ] Cuadro guardado como tabla de resultados.
7. [ ] *(Flujo B)* Secret scope + `prueba_de_humo()` verde (Fase 3).
8. [ ] *(Flujo B)* RAG reconstruido (`--status` con chunks > 0) (Fase 5).
9. [ ] *(Flujo B)* Una validación completa → informe en Volume (Fase 7).
10. [ ] Job programado + revisión de seguridad/permisos (Fase 8).

17. Qué aprender y a quién pedir ayuda

- **Aprende (lo justo):** cómo leer una tabla con `spark.read.table().toPandas()`, qué es un secret scope, y cómo se crea un Job. Con eso haces el 90%.
- **Pide ayuda a: plataforma/datos** (accesos, UC, cluster, Volume), **seguridad** (egress al LLM), y quien conozca el **esquema de las tablas** (para mapear las columnas a los `params` de los controles).
- **No te saltes** la §5–6: empezar pequeño (un control, una tabla) evita el error clásico de primerizo de querer automatizar todo antes de que **una** cosa funcione.

Resumen en una línea

Es **factible** y el diseño ayuda (RAG local, controles en pandas). Para tu primera producción: **empieza por el Flujo A** (controles sobre tablas, ~1–2 semanas, sin sacar datos a la IA); el Flujo B (cuestionamiento) es igual de viable pero **depende de que seguridad apruebe el egress** al proveedor de IA. El único trabajo de código nuevo de peso es **un adaptador** que lee tablas en vez de CSV.